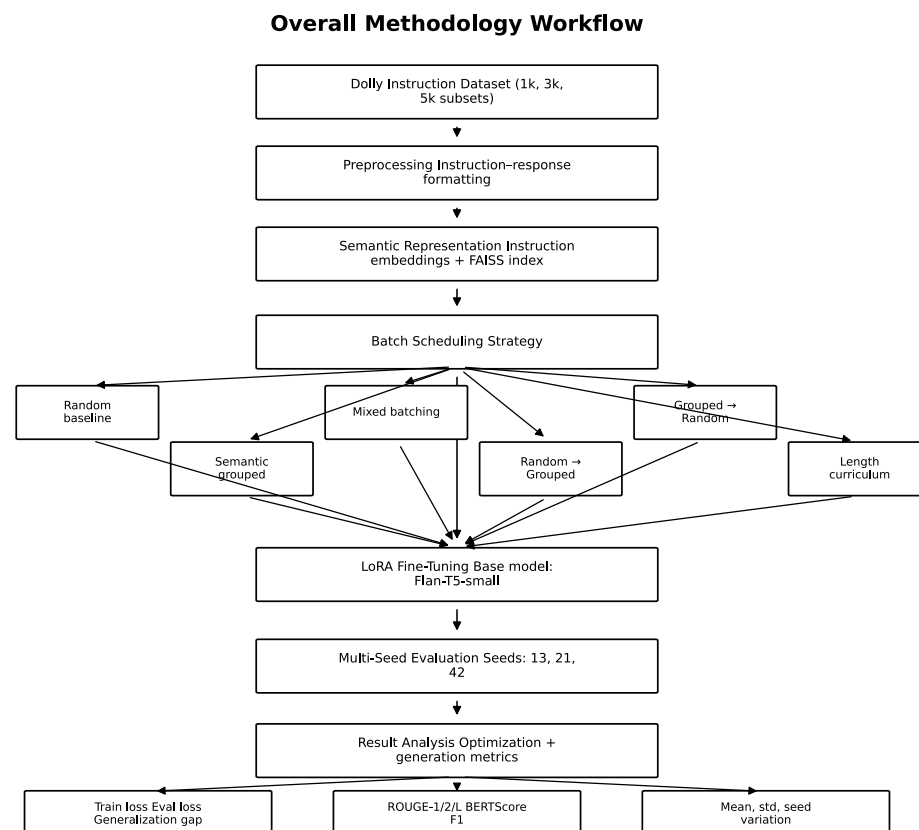




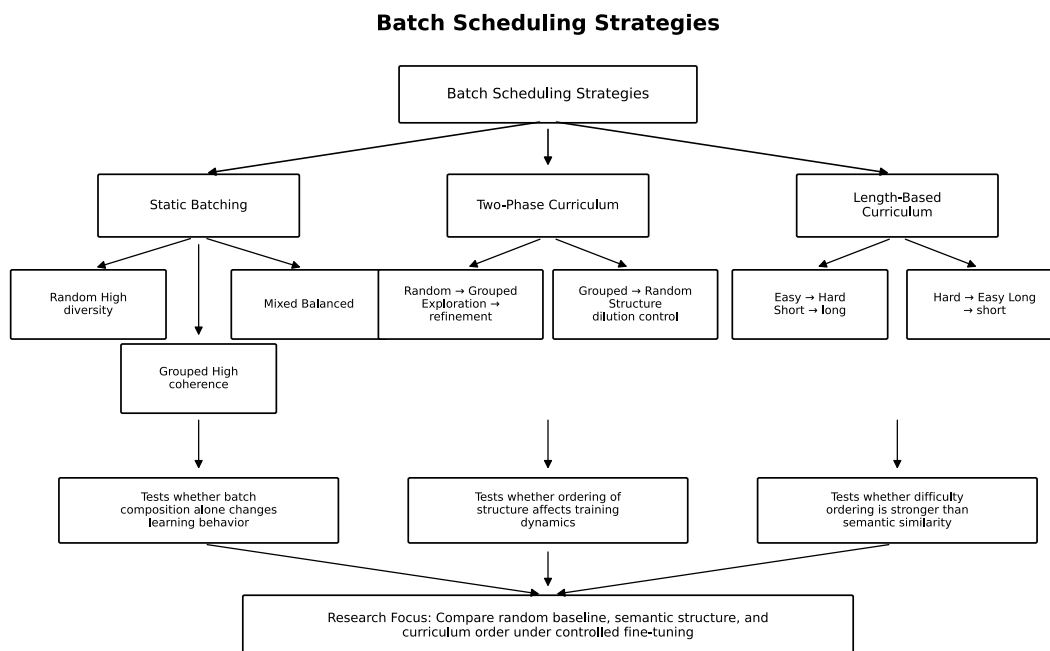
Chapter 3: Methodology

Created by	 Aroosh Ahmad
Created time	May 18, 2026 8:07 AM
Last edited by	 Aroosh Ahmad
Last updated time	May 18, 2026 8:21 AM

▼ All Figures



Figure_3_1_Overall_Methodology_Workflow



Figure_3_2_Batch_Scheduling_Strategies

3.1 Overview

This chapter describes the methodology used to investigate the effect of batch scheduling strategies on instruction fine-tuning of large language models. The study focuses on whether mini-batch construction and ordering can influence optimization behavior, generalization, and generation quality during supervised instruction fine-tuning.

The methodology is designed around a controlled comparison between standard random batching and structured alternatives, including semantic grouping, mixed batching, two-phase curriculum scheduling, and length-based difficulty curriculum. The main objective is not to propose a new model architecture, but to examine whether changing the organization of training samples can affect fine-tuning behavior.

The study follows an empirical research design. A base instruction-following model is fine-tuned under different batch scheduling strategies, and the resulting

models are compared using optimization metrics and generation metrics. The experiments are conducted across multiple dataset sizes and random seeds to reduce the risk of drawing conclusions from a single run.

3.2 Research Design

The research is structured as a comparative experimental study. The central variable is the **batch scheduling strategy**, while the model family, fine-tuning method, dataset source, and evaluation protocol are kept consistent across comparable experiments.

The study compares five main families of training strategies:

1. **Random batching**, used as the baseline.
2. **Semantic grouped batching**, where semantically similar instructions are grouped into the same batch.
3. **Mixed batching**, where random and semantically grouped samples are combined.
4. **Two-phase curriculum scheduling**, where the batching strategy changes between training phases.
5. **Length-based curriculum scheduling**, where instruction length is used as a proxy for difficulty.

The research design is guided by three hypotheses. First, semantically similar samples may produce more coherent gradient updates. Second, the order in which batching strategies are applied may influence the training trajectory. Third, difficulty-based curriculum may provide a stronger signal than semantic similarity alone.

This design allows the study to separate two related but different questions: whether **batch composition** matters, and whether **batch ordering** matters.

Insert Figure 3.1 Here

Figure 3.1: Overall Methodology Workflow

Use this diagram in your thesis:

▼ Figure

fig-3.1.png

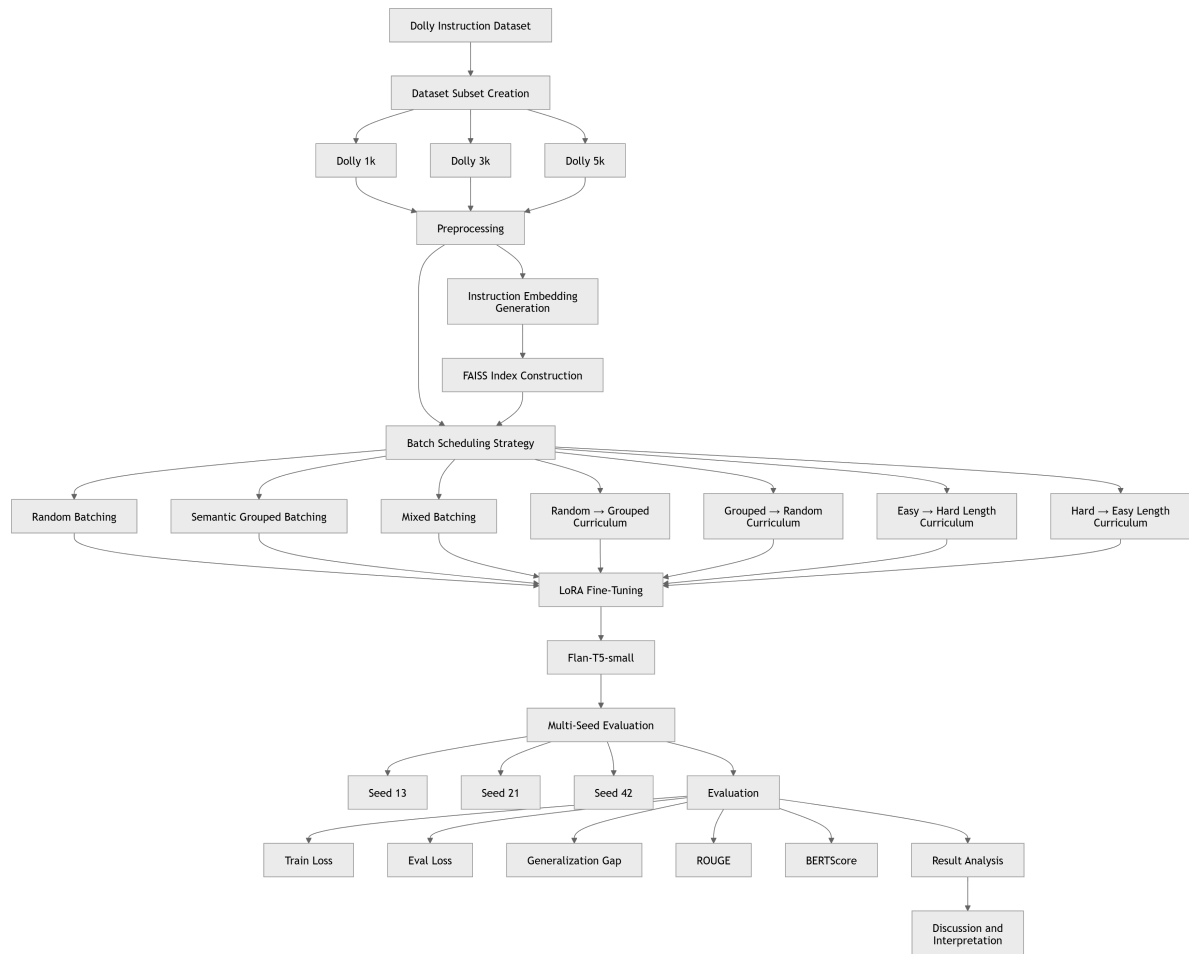
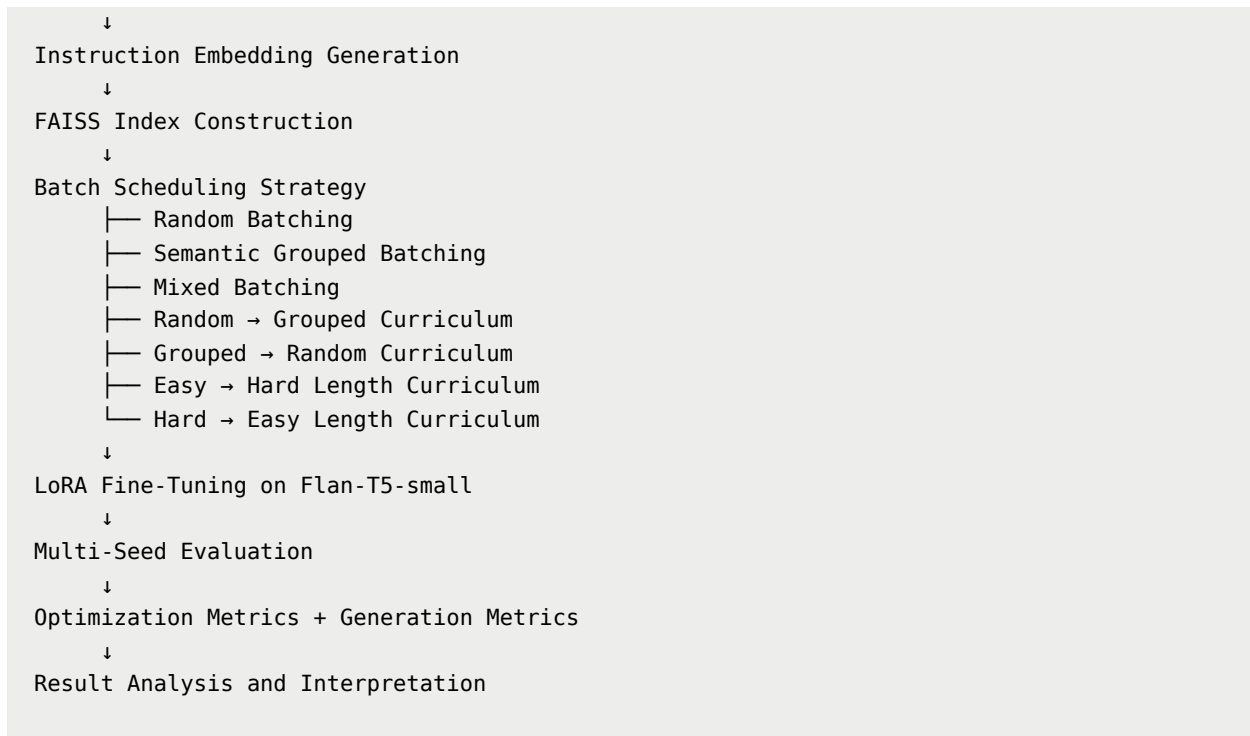


Figure 3.1: Overall methodology workflow for evaluating batch scheduling strategies in instruction fine-tuning. The pipeline begins with Dolly dataset subset creation, followed by preprocessing, semantic embedding generation, batch scheduling, LoRA fine-tuning of Flan-T5-small, multi-seed evaluation, and final result analysis.

Dolly Dataset
↓
Preprocessing and Subset Creation



This should be converted into a clean flowchart later. It belongs in Methodology, not Results.

3.3 Dataset Preparation

The experiments use subsets of the Dolly instruction-following dataset. The dataset contains instruction–response examples from multiple task categories, including question answering, summarization, classification, information extraction, brainstorming, and open-ended generation. This diversity makes the dataset suitable for evaluating instruction fine-tuning behavior.

Three dataset scales are used:

- **Dolly 1k** for initial and controlled pilot experiments.
- **Dolly 3k** for intermediate scaling experiments.
- **Dolly 5k** for final validation and curriculum experiments.

The purpose of using multiple dataset sizes is to observe whether the effect of batching strategies changes as the amount of training data increases. Smaller subsets make early experimentation faster, while larger subsets provide a more reliable test of whether an observed pattern generalizes beyond pilot runs.

Each example is treated as an instruction–response pair. The instruction text is used both for fine-tuning and, in the case of semantic batching, for computing semantic embeddings.

3.4 Base Model and Fine-Tuning Method

The base model used in this study is **Flan-T5-small**, an instruction-tuned variant of the T5 architecture. T5 frames natural language processing tasks as text-to-text problems, making it suitable for instruction–response fine-tuning [2]. Flan-T5 extends this direction through instruction fine-tuning across diverse tasks [3].

Fine-tuning is performed using **Low-Rank Adaptation (LoRA)**. LoRA freezes the main pre-trained model weights and introduces trainable low-rank matrices into selected layers, reducing the number of trainable parameters while still allowing the model to adapt to the target task [5].

LoRA is appropriate for this study because the research requires multiple fine-tuning runs across batching strategies, dataset sizes, and random seeds. Full fine-tuning would be computationally more expensive and less practical. Using LoRA makes the experimental comparison feasible while keeping the fine-tuning approach consistent across experiments.

3.5 Baseline: Random Batching

Random batching is used as the primary baseline. In this strategy, training samples are randomly shuffled and divided into mini-batches. This is the standard approach in most supervised fine-tuning pipelines.

Random batching has several useful properties. It exposes the model to diverse examples within each training phase, reduces systematic ordering bias, and provides stochastic gradient estimates from mixed regions of the data distribution. In heterogeneous instruction-tuning datasets, this diversity may be especially important because each batch can contain examples from different task types.

Formally, given a training dataset:

$$D = \{(x_i, y_i)\}_{i=1}^N \quad D = \{(x_i, y_i)\}_{i=1}^N$$

$$D = \{(x_i, y_i)\}_{i=1}^N$$

where x_i represents the instruction and y_i represents the target response, random batching constructs each mini-batch B_t by randomly sampling examples from D :

$$B_t \subset D$$

$$B_t \subset D$$

The random batching baseline is necessary because the study aims to test whether structured alternatives provide any meaningful advantage over standard practice.

3.6 Semantic Mini-Batch Construction

Semantic batching is designed to group semantically similar instructions into the same mini-batch. The intuition is that related instructions may produce more aligned gradients, which could potentially improve learning stability, convergence, or generalization.

The semantic batching process consists of four steps.

3.6.1 Instruction Embedding

Each instruction is converted into a dense vector representation using a sentence embedding model. Sentence embeddings are useful because they represent sentence-level meaning in vector space, allowing semantically similar instructions to be located near one another [6].

Let x_i be an instruction. Its embedding is computed as:

$$e_i = f(x_i)$$

$$e_i = f(x_i)$$

where f is the sentence embedding model and e_i is the resulting vector representation.

3.6.2 FAISS Index Construction

After generating embeddings for all instructions, the vectors are indexed using FAISS. FAISS enables efficient similarity search over dense vector collections and is suitable for nearest-neighbor retrieval at scale [7].

The embedding index is used to retrieve semantically similar instructions for each sample.

3.6.3 Nearest-Neighbor Retrieval

For each instruction embedding e_{ie_i} , the nearest neighbors are retrieved from the FAISS index. These neighbors represent examples that are semantically close to the current instruction.

The neighbor set can be represented as:

$$N_k(x_i) = \{x_{j_1}, x_{j_2}, \dots, x_{j_k}\}$$

$$N_k(x_i) = \{x_{j_1}, x_{j_2}, \dots, x_{j_k}\}$$

where $N_k(x_i)$ denotes the top k nearest instructions to x_i .

3.6.4 Grouped Batch Formation

Mini-batches are formed by grouping semantically related examples together. Each grouped batch therefore contains examples that are close in embedding space.

This strategy increases within-batch semantic coherence but reduces within-batch diversity. The central question of the thesis is whether this trade-off improves instruction fine-tuning or whether random diversity remains more beneficial.

Algorithm 3.1

Algorithm 3.1: Semantic Mini-Batch Construction

```
Input:
  Dataset  $D = \{(x_i, y_i)\}$ 
  Sentence embedding model  $f$ 
  Batch size  $b$ 
  FAISS index  $I$ 

Output:
  Semantically grouped mini-batches

Steps:
1. For each instruction  $x_i$  in  $D$ :
   compute embedding  $e_i = f(x_i)$ 
```


2. Add all embeddings $\{e_i\}$ to FAISS index I
3. Initialize an empty list of batches
4. While unassigned examples remain:
 - select an unassigned instruction x_i
 - retrieve nearest neighbors $N_k(x_i)$ from I
 - construct batch B using x_i and nearest unassigned neighbors
 - mark selected examples as assigned
 - add B to batch list
5. Return grouped batches

Algorithm 3.1: Semantic Mini-Batch Construction

Input:

Dataset $D = \{(x_i, y_i)\}$
 Sentence embedding model f
 Batch size b
 FAISS index I

Output:

Semantically grouped mini-batches

Steps:

1. For each instruction x_i in D :
 - compute embedding $e_i = f(x_i)$
2. Add all embeddings $\{e_i\}$ to FAISS index I
3. Initialize an empty list of batches
4. While unassigned examples remain:
 - select an unassigned instruction x_i
 - retrieve nearest neighbors $N_k(x_i)$ from I
 - construct batch B using x_i and nearest unassigned neighbors

```
mark selected examples as assigned
add B to batch list
```

5. Return grouped batches

Algorithm 3.1: Semantic mini-batch construction using instruction embeddings and FAISS nearest-neighbor retrieval.

3.7 Mixed Batching

Mixed batching is introduced as an intermediate strategy between fully random and fully grouped batching. The purpose of this strategy is to balance two competing properties:

- **Diversity**, provided by random sampling.
- **Coherence**, provided by semantic grouping.

In mixed batching, each mini-batch contains a combination of randomly sampled examples and semantically related examples. This approach is motivated by the possibility that fully grouped batches may become too narrow, while fully random batches may lack focused semantic structure.

Mixed batching therefore tests whether a balance between diversity and structure can outperform either extreme.

3.8 Curriculum-Based Batch Scheduling

Curriculum learning studies whether the order of training examples affects learning. Bengio et al. introduced curriculum learning as a training strategy in which examples are presented in a meaningful order, often from easier to harder samples [8].

In this thesis, curriculum is applied at the level of batch scheduling. Instead of keeping one batching strategy fixed throughout training, the training process is divided into phases, and the batching strategy changes between phases.

The two main curriculum strategies are:

3.8.1 Random → Grouped Curriculum

In this strategy, training begins with random batching and later switches to semantic grouped batching.

The motivation is an **exploration-refinement** pattern:

- **Phase 1: Random batching**

The model is exposed to diverse instruction types and learns broad task behavior.

- **Phase 2: Grouped batching**

The model receives more semantically focused updates, which may refine learned representations.

This strategy tests whether semantic structure is more useful after the model has already learned a broad representation of the instruction space.

3.8.2 Grouped → Random Curriculum

This strategy reverses the order. Training begins with semantically grouped batches and later switches to random batching.

This acts as a control condition for testing whether the direction of ordering matters. If both strategies perform similarly, then ordering may not be important. If they differ, then the sequence of data exposure has an effect on training dynamics.

The reviewed experiment notes indicate that this comparison became a central part of the thesis because it helped shift the research question from “Does semantic grouping help?” to “When should semantic structure be introduced during training?”

3.9 Length-Based Difficulty Curriculum

In addition to semantic curriculum, this thesis evaluates a simple difficulty-based curriculum. Instruction length is used as a proxy for difficulty. The assumption is that longer instructions may contain more constraints, more context, or more complex requirements than shorter instructions.

Two length-based schedules are evaluated:

3.9.1 Easy → Hard Curriculum

In the easy-to-hard strategy, shorter instructions are used earlier in training, followed by longer instructions later. This follows the classical curriculum learning intuition that models may benefit from learning simpler examples first before progressing to more complex ones [8].

3.9.2 Hard → Easy Curriculum

In the hard-to-easy strategy, longer instructions are used earlier, followed by shorter instructions later. This reverse curriculum is included to test whether early exposure to complex examples improves representation learning before later stabilization on simpler examples.

This strategy is important because the experimental results later show that difficulty-based ordering provides a stronger signal than semantic grouping alone. The reviewed thesis findings identify hard-to-easy length-based scheduling as the strongest curriculum direction, although improvements remain modest and must be interpreted conservatively.

3.10 Summary of Batch Scheduling Strategies

Table 3.1: Summary of Batch Scheduling Strategies

Strategy	Description	Purpose
Random	Samples are shuffled and batched randomly	Baseline comparison
Grouped	Semantically similar instructions are placed together	Tests semantic coherence
Mixed	Random and grouped samples are combined	Balances diversity and coherence
Random → Grouped	Random batching followed by grouped batching	Tests exploration → refinement
Grouped → Random	Grouped batching followed by random batching	Tests structure dilution / anti-curriculum
Easy → Hard Length	Short instructions followed by longer instructions	Tests classical difficulty curriculum

Strategy	Description	Purpose
Hard → Easy Length	Long instructions followed by shorter instructions	Tests reverse difficulty curriculum

This table belongs in Methodology because it explains the design. It is not a results table.

Figure 3.2: Batch Scheduling Strategies

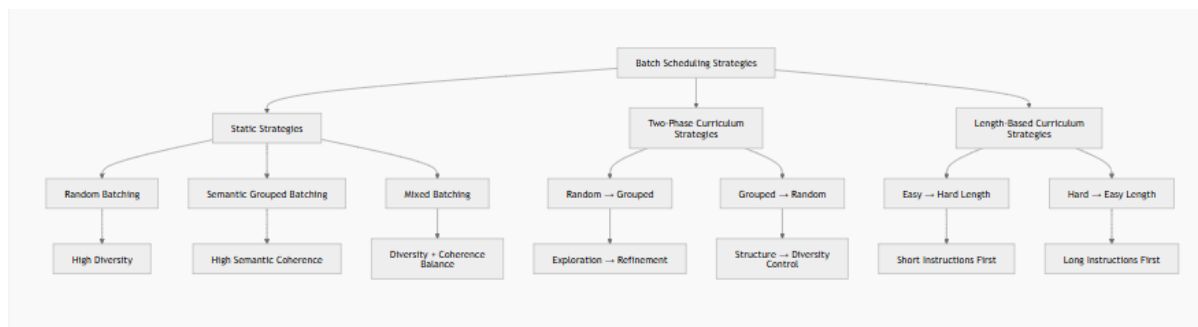


Figure 3.2: Categorization of batch scheduling strategies used in this study. The strategies are divided into static batching, two-phase curriculum scheduling, and length-based curriculum scheduling.

3.11 Experimental Stages

The experiments are organized in stages to move from exploratory analysis toward controlled validation.

3.11.1 Exploratory Stage

The initial experiments evaluate whether batch properties can be manipulated successfully. Random, grouped, and mixed batching are compared to observe whether semantic grouping changes batch coherence and training behavior.

These early experiments help verify that the batching mechanism works as intended. However, early observations are treated cautiously because later multi-seed experiments provide a more reliable basis for conclusions.

3.11.2 Controlled Pilot Stage

The controlled pilot experiments compare random, grouped, random-to-grouped, and grouped-to-random strategies across multiple seeds. This stage isolates the effect of ordering and provides the first structured test of the curriculum hypothesis.

The experiment inventory records these controlled experiments as exp_012 to exp_015, using Flan-T5-small with LoRA and seeds 13, 21, and 42.

3.11.3 Extended Training Stage

A longer training stage is conducted to evaluate whether observed patterns persist when training is extended. This stage is important because some effects may not appear clearly in short training runs.

3.11.4 Scaling Stage

The experiments are then scaled from 1k to 3k and 5k samples. This tests whether the impact of batching strategies changes with dataset size. The reviewed findings indicate that batching effects become smaller as dataset size increases, while curriculum signals remain more visible in the length-based experiments.

3.11.5 Length Curriculum Stage

The final stage evaluates easy-to-hard and hard-to-easy length-based curriculum strategies on the 5k dataset. This stage directly tests whether difficulty-aware ordering provides a stronger signal than semantic similarity-based grouping.

3.12 Multi-Seed Evaluation Protocol

To reduce the risk of seed-specific conclusions, key experiments are repeated using three random seeds:

$\{13, 21, 42\} \setminus \{13, 21, 42\}$

$\{13, 21, 42\}$

Multi-seed evaluation is important because small differences between batching strategies may be caused by random initialization, data ordering, or stochastic training behavior. By averaging across multiple seeds, the study obtains a more reliable estimate of method performance.

For each method, the mean and standard deviation are computed across seeds. This allows the study to compare not only average performance but also stability. The reviewed findings emphasize that many method differences are small and often comparable to seed-induced variation, which limits the strength of performance claims.

3.13 Evaluation Methodology

The evaluation protocol includes both optimization-based and generation-based metrics.

3.13.1 Optimization Metrics

The optimization metrics are:

- **Training loss**
- **Evaluation loss**
- **Generalization gap**
- **Phase-wise evaluation improvement**

Training loss measures how well the model fits the training data, while evaluation loss measures performance on held-out data. The generalization gap is used to compare training and evaluation behavior. Phase-wise evaluation improvement is especially important for two-phase curriculum experiments because it helps measure how performance changes after switching strategies.

3.13.2 Generation Metrics

The generation metrics are:

- **ROUGE-1**
- **ROUGE-2**
- **ROUGE-L**
- **BERTScore F1**

ROUGE evaluates lexical overlap between generated and reference text [9].

BERTScore uses contextual embeddings to evaluate semantic similarity between

generated and reference text [10].

These metrics are included because optimization loss alone may not fully reflect instruction-following quality. A batching strategy may not improve evaluation loss but could still affect generated outputs. Therefore, both loss-based and generation-based evaluation are necessary.

The reviewed results later show that generation differences across batching strategies are very small, supporting the need for conservative interpretation.

3.14 Validity and Reliability Considerations

Several design choices are used to improve the reliability of the study.

First, random batching is treated as a strong baseline rather than a weak default. This prevents overclaiming the value of structured batching.

Second, multi-seed experiments are used to reduce dependence on individual training runs.

Third, experiments are conducted across multiple dataset sizes to observe whether effects persist as data scale increases.

Fourth, both optimization and generation metrics are used. This reduces the risk of drawing conclusions from only one type of metric.

Finally, weak or unsupported claims are avoided. The study does not assume that semantic batching improves performance. Instead, it tests the hypothesis empirically and interprets the findings conservatively.

The reviewed thesis summary explicitly notes that semantic batching does not significantly improve performance, random batching remains a robust baseline, and improvements from batching strategies are small in magnitude.

3.15 Summary

This chapter presented the methodology used to study batch scheduling strategies for instruction fine-tuning. The study compares random batching, semantic grouped batching, mixed batching, two-phase curriculum scheduling, and length-based difficulty curriculum. Semantic grouping is implemented using sentence embeddings and FAISS-based nearest-neighbor retrieval, while

curriculum scheduling is evaluated through random-to-grouped, grouped-to-random, easy-to-hard, and hard-to-easy variants.

The methodology is designed to isolate the effect of batch composition and batch ordering while keeping the model, fine-tuning approach, and evaluation protocol consistent across comparable experiments. The next chapter describes the experimental setup, including dataset configuration, model details, training environment, experiment groups, and reproducibility settings.

References Used in Chapter 3

Use the same reference numbering from Chapter 2:

- [2] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, 2020.
- [3] H. W. Chung et al., "Scaling Instruction-Finetuned Language Models," *Journal of Machine Learning Research*, 2024.
- [5] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations*, 2022.
- [6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *EMNLP-IJCNLP*, 2019.
- [7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, 2019.
- [8] Y. Bengio et al., "Curriculum Learning," *ICML*, 2009.
- [9] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," *ACL Workshop*, 2004.
- [10] T. Zhang et al., "BERTScore: Evaluating Text Generation with BERT," *ICLR*, 2020.